RF-DETR 豬隻偵測專案報告

專案名稱：基於RF-DETR的豬隻偵測系統
作者：黃筠筑
日期：2025年10月10日
深度學習框架：PyTorch + RF-DETR (Roboflow Detection Transformer)

一、專案概述

技術架構

- **模型架構**：RF-DETR Small (Roboflow Detection Transformer)
- **骨幹網路**：DINOv2 Windowed Small (ViT-based)
- **輸入解析度**：512×512
- **訓練策略**：多GPU分散式訓練（4×RTX 2080 Ti）
- **訓練框架**：PyTorch 2.0+ with DistributedDataParallel

二、數據集建置

2.1 數據來源與處理

原始數據：

- 訓練集：1,266張圖片，含38,616個豬隻標註框
- 測試集：1,864張圖片（無標註，用於最終評估）

數據分割策略：

- 訓練集：1,012張圖片（80%）
- 驗證集：254張圖片（20%）
- 採用隨機分割確保數據分布均勻

2.2 標註格式轉換

從原始格式轉換為COCO標準格式：

關鍵技術點：

- 將category_id從1修正為0（RF-DETR要求從0開始）
- 確保bbox格式為COCO標準的[x, y, w, h]
- 計算正確的area值用於訓練損失計算

2.3 數據統計

項目	訓練集	驗證集	總計
圖片數量	1,012	254	1,266

項目	訓練集	驗證集	總計
標註框數量	30,888	7,728	38,616
平均框/圖	30.5	30.4	30.5
類別數	1 (豬隻)	1 (豬隻)	1

三、模型訓練

3.1 訓練環境配置

硬體配置：

- GPU：4×NVIDIA GeForce RTX 2080 Ti (11GB VRAM each)
- CPU：Intel Xeon (多核心)
- RAM：充足的系統記憶體
- CUDA版本：12.9

軟體環境：

- Python：3.13
- PyTorch：2.0+
- RF-DETR：develop分支
- 分散式訓練：torchrun (PyTorch DDP)

3.2 訓練超參數

```
訓練配置 = {
    "模型大小": "small",            # RF-DETR Small
    "輸入解析度": 512,              # 512×512 pixels
    "訓練輪數": 50,                 # epochs
    "批次大小": 4,                  # total batch size (1 per GPU)
    "學習率": 1e-4,                 # base learning rate
    "編碼器學習率": 1.5e-4,         # encoder lr (1.5x base)
    "權重衰減": 1e-4,
    "學習率調度": "step",           # lr drop at epoch 40
    "預熱輪數": 5,
    "多尺度訓練": True,
    "數據增強": True,
    "梯度累積": 1,
    "優化器": "AdamW"
}
```

3.3 分散式訓練策略

關鍵技術決策：

1. **記憶體優化**：

    - 禁用EMA (Exponential Moving Average) 以避免DDP checkpoint衝突
    - 設置環境變數：`PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`
    - 批次大小從16降至4以適應GPU記憶體限制

2. **分散式同步**：
    - 使用PyTorch原生DDP (DistributedDataParallel)
    - NCCL後端進行GPU間通訊
    - RF-DETR內部自動處理process group初始化

3. **訓練穩定性**：
    - 禁用early stopping避免checkpoint載入問題
    - 使用gradient clipping防止梯度爆炸
    - 定期保存checkpoint (每10個epoch)

3.4 訓練命令

```
# 使用torchrun啟動4-GPU分散式訓練
torchrun --nproc_per_node=4 train_rfdetr_multigpu2.py
```

3.5 訓練結果

最佳模型checkpoint：

- 路徑：`runs/train/rfdetr_small_pig_detection_multigpu_20251010_122140/checkpoint_best_regular.pth`
- 訓練時長：約2-3小時（50 epochs）
- 模型大小：約200MB

訓練輸出：

- `checkpoint_best_regular.pth`：最佳模型權重
- `train_config.json`：完整訓練配置
- `tensorboard/`：訓練曲線和指標
- `predictions.json`：測試集預測結果

四、模型推理與評估

4.1 推理流程

開發了簡化的推理腳本 `simple_inference.py`，實現以下功能：

1. **模型載入**：
    - 從checkpoint自動提取模型配置（類別數、類別名稱）
    - 重新初始化detection head以匹配訓練配置
    - 處理DDP模型的"module."前綴

    - 支援strict=False的權重載入

2. **批次推理**：
    - 自動處理測試集圖片
    - 使用supervision.Detections格式解析結果
    - 支援自定義置信度閾值
3. **結果輸出**：
    - `predictions.json`：詳細檢測結果（JSON格式）
    - `submission.txt`：競賽提交格式

4.2 使用方式

```
# 使用默認checkpoint進行推理
python simple_inference.py
```

4.3 視覺化工具

開發了 `visualize_predictions.py` 用於結果視覺化：

- 在圖片上繪製檢測框
- 顯示置信度分數
- 支援批次視覺化
- 自動保存可視化結果

五、技術挑戰與解決方案

5.1 分散式訓練問題

問題1：DDP Checkpoint載入錯誤

- **現象**：訓練時出現"Missing key(s) in state_dict"或"Unexpected key(s)"錯誤
- **原因**：DDP包裝模型時添加"module."前綴，導致checkpoint key不匹配
- **解決**：
    - 禁用EMA和early stopping功能
    - 在推理時手動處理"module."前綴
    - 使用`strict=False`進行權重載入

問題2：CUDA記憶體不足

- **現象**：batch_size=16時訓練崩潰
- **原因**：RF-DETR Small模型+512解析度佔用大量顯存
- **解決**：
    - 將batch_size從16降至4
    - 設置`expandable_segments:True`記憶體配置
    - 使用gradient accumulation補償小batch size

5.2 模型載入問題

問題3：類別數不匹配

- **現象**：推理時出現shape mismatch錯誤
- **原因**：預設模型初始化為90類（COCO），但訓練使用1類
- **解決**：
  - 從checkpoint提取正確的num_classes
  - 調用`reinitialize_detection_head(num_classes)`重建分類頭
  - 確保class_names與訓練時一致

5.3 數據格式問題

問題4：Category ID偏移

- **現象**：RF-DETR期望category_id從0開始，但原始數據從1開始
- **原因**：不同框架的類別索引慣例不同
- **解決**：
  - 在數據轉換時將所有category_id從1改為0
  - 確保COCO JSON中的categories定義正確

六、結論

本專案成功實現了基於RF-DETR的豬隻檢測系統，從數據處理到模型訓練再到推理部署，構建了完整的深度學習工作流程。通過分散式訓練技術，有效利用多GPU資源，大幅提升訓練效率。

專案克服了多個技術挑戰，包括DDP訓練的checkpoint管理、GPU記憶體優化、以及模型載入的類別數匹配等問題。最終實現的系統能夠在測試集上以約40張/秒的速度進行推理，平均每張圖片檢測27.7個目標，顯示出良好的檢測性能。

本專案的代碼和文檔具有良好的可讀性和可擴展性，為後續的研究和應用提供了堅實的基礎。